Faculté des sciences ben m'sick
Université Hassan II de Casalanca

Université Hassan 2 er
Faculté des Sciences ben m'sik Casablanca

Big Data & Data Science

Projet réalisé dans le cadre du module DevOps

Filière : Big Data & Data Science

Mise en œuvre d'une chaîne CI/CD pour le déploiement d'une application conteneurisée sur AWS avec Kubernetes

Soutenue à la FACULTE Ben M'sik de CASABLANCA, le 24/03/2026

Soutenu par :

MSOUHLI RAHIL

ISRAA SEKKAT

Devant le jury composé de

Pr. Benabbou Fouzia Encadrant

Résumé

Ce projet a pour objectif la conception et la mise en œuvre d'une chaîne DevOps complète permettant d'automatiser le cycle de vie d'une application, depuis son développement jusqu'à son déploiement sur une infrastructure cloud. Dans un contexte où les exigences en matière de rapidité, de fiabilité et de reproductibilité sont de plus en plus élevées, l'adoption des pratiques DevOps constitue un levier essentiel pour améliorer la qualité des systèmes informatiques.

L'application utilisée dans ce projet est une API REST légère développée en Python avec le framework Flask. Cette application a été conteneurisée à l'aide de Docker afin de garantir la portabilité et la cohérence de son environnement d'exécution. Un pipeline d'intégration et de déploiement continu (CI/CD) a été mis en place à l'aide de GitHub Actions, permettant d'automatiser les phases de test, de construction de l'image Docker, de publication sur Docker Hub et de déploiement sur Kubernetes.

L'infrastructure cloud a été provisionnée sur la plateforme AWS en utilisant Terraform, suivant le principe de l'Infrastructure as Code, afin d'assurer une gestion déclarative, versionnée et reproductible des ressources. La configuration des machines ainsi que la mise en place du cluster Kubernetes (K3s) ont été automatisées grâce à Ansible, ce qui a permis de réduire les interventions manuelles et d'assurer la cohérence de l'environnement.

Le déploiement de l'application a été réalisé sur Kubernetes à l'aide de manifests YAML définissant un Deployment et un Service de type NodePort, permettant d'assurer la disponibilité et l'accessibilité de l'application. Les résultats obtenus montrent une intégration cohérente des différents outils DevOps et une automatisation efficace des principales étapes du processus.

Ce projet met en évidence l'importance de la conteneurisation, de l'automatisation et de l'orchestration dans la mise en place d'architectures modernes. Il ouvre également des perspectives d'amélioration, notamment en renforçant certains aspects du déploiement continu et en intégrant des outils de monitoring pour une meilleure supervision du système.

Abstract

This project aims to design and implement a complete DevOps pipeline that automates the lifecycle of an application, from development to deployment on a cloud infrastructure. In a context where requirements for speed, reliability, and reproducibility are continuously increasing, adopting DevOps practices has become essential to improve the quality of software systems.

The application used in this project is a lightweight REST API developed in Python using the Flask framework. It was containerized using Docker to ensure portability and consistency across different execution environments. A CI/CD pipeline was implemented using GitHub Actions, enabling automated testing, Docker image building, pushing to Docker Hub, and deployment to Kubernetes.

The cloud infrastructure was provisioned on AWS using Terraform, following the Infrastructure as Code approach, which ensures a declarative, version-controlled, and reproducible environment. The configuration of the machines and the setup of the Kubernetes cluster (K3s) were automated using Ansible, minimizing manual intervention and ensuring consistency across nodes.

The application was deployed on Kubernetes using YAML manifests defining a Deployment and a NodePort Service, ensuring availability and external access. The results demonstrate a coherent integration of DevOps tools and an effective automation of the main stages of the development and deployment process.

This project highlights the importance of containerization, automation, and orchestration in building modern and scalable architectures. It also opens perspectives for further improvements, particularly in enhancing certain aspects of continuous deployment and integrating monitoring solutions for better system observability.

Liste des figures

FIGURE 1-ARTCHITECTURE DU PROJET 16
FIGURE 2-PAGE D'ACCUEIL DE L'APPLICATION WEB 26
FIGURE 3-INTERFACE DE NAVIGATION DES PLAYLISTS ET CONTENUS MUSICAUX 27
FIGURE 4-VERIFICATION DU CONTENEUR DOCKER EN COURS D'EXECUTION AVEC LA COMMANDE 28
FIGURE 5-IMAGE DOCKER PUBLIEE SUR DOCKER HUB 28
FIGURE 6-CONFIGURATION DES SECRETS GITHUB ACTIONS POUR L'AUTHENTIFICATION DOCKER HUB 30
FIGURE 7-EXECUTION REUSSIE DU PIPELINE CI/CD DANS GITHUB ACTIONS 30
FIGURE 8-CONFIGURATION DES SECRETS DU DEPOT DANS GITHUB ACTIONS 31
FIGURE 9-RESULTAT DE LA COMMANDE TERRAFORM AFFICHANT LES ADRESSES IP PUBLIQUES ET PRIVEES DES INSTANCES EC2 . 32
FIGURE 10-AFFICHAGE DES ADRESSES IP PUBLIQUES DES INSTANCES EC2 GENEREES PAR TERRAFORM 32
FIGURE 11-INSTANCES EC2 CREEES SUR AWS POUR LE CLUSTER KUBERNETES (MASTER ET WORKER) 33
FIGURE 12-VERIFICATION DE LA CONNECTIVITE DES NŒUDS AVEC ANSIBLE (COMMANDE PING) 34
FIGURE 13-VERIFICATION DETAILLEE DES NŒUDS DU CLUSTER K3S AVEC LEURS ADRESSES IP ET LEUR ETAT 35
FIGURE 14-EXECUTION DU PLAYBOOK ANSIBLE POUR L'INSTALLATION DU CLUSTER K3S (MASTER ET WORKER) 35
FIGURE 15-VERIFICATION DES PODS ET DU SERVICE KUBERNETES AVEC LA COMMANDE KUBECTL 37
FIGURE 16-STRUCTURE DU DEPOT GITHUB DU PROJET DEVOPS 40
FIGURE 17-EXECUTION REUSSIE DU PIPELINE CI/CD SUR GITHUB ACTIONS- 46
FIGURE 18-DETAIL DES ETAPES DU PIPELINE CI/CD 46
FIGURE 19-DETAIL DES ETAPES DU PIPELINE CI/CD 46

Liste des tableaux

TABLEAU 1-TABLEAU RECAPITULATIF DES OUTILS / TECHNOLOGIES 21
TABLEAU 2- DESCRIPTION DES ENDPOINTS DE L'API FLASK 26
TABLEAU 3-DESCRIPTION DES ETAPES DU PIPELINE CI/CD 29
TABLEAU 4-PORTS ET FLUX RESEAU UTILISES DANS LE CLUSTER KUBERNETES 32

Table des matières

Introduction générale 9
CHAPITRE 1 : CONTEXTE GÉNÉRAL 10
Introduction 11
1. Contexte général du projet 11
2. Problématique 11
3. Objectifs du projet 12
a. Objectif général 12
b. Objectifs spécifiques 12
Conclusion 13
CHAPITRE 2 : Architecture du Projet 14
Introduction 15
1. Vue globale de l'architecture 15
2. Description des différentes couches 16
a. Couche Application 16
b. Couche CI/CD (GitHub Actions) 17
c. Couche Conteneurisation (Docker) 17
d. Couche Infrastructure (Terraform / AWS) 17
e. Couche Configuration (Ansible) 18
f. Couche Orchestration (Kubernetes K3s) 18
3. Flux de fonctionnement global 18
Conclusion 19
CHAPITRE 3 : Outils et Technologies Utilisés 19
Introduction 21
1. Tableau récapitulatif des outils / technologies 21
2. Justification des choix technologiques 22
Conclusion 23
Chapitre 4 : Conception et Mise en œuvre de la solution 24
Introduction 25
1. Développement de l'application et conteneurisation 25
a. Présentation fonctionnelle de l'application 25
b. Tests unitaires 27

c. Conteneurisation avec Docker 27
2. Intégration continue (CI/CD) 28
a. Mise en place du pipeline 28
d. Analyse du pipeline 31
3. Provisionnement de l'infrastructure avec Terraform 31
a. Ressources définies 32
b. Analyse critique de la configuration Terraform 33
4. Configuration automatisée avec Ansible et mise en place du cluster K3s 33
a. Inventaire Ansible 33
b. Installation du master K3s 34
c. Récupération du token et jonction du worker 34
d. Analyse critique du playbook 35
e. Emplacement des livrables Ansible 36
5. Déploiement de l'application sur Kubernetes 36
a. Analyse du Deployment 36
b. Analyse du Service 36
c. Processus type de déploiement 36
d. Monitoring optionnel 37
e. Emplacement des livrables Kubernetes 37
Conclusion 37
Chapitre 5 : Documentation pour la reproduction de l'infrastructure 38
Introduction 39
1. Prérequis 39
2. Préparation du projet 39
3. Provisionnement de l'infrastructure avec Terraform 40
4. Configuration de l'inventaire Ansible 40
5. Installation et configuration du cluster Kubernetes (K3s) 41
6. Vérification du cluster 41
7. Test local de l'application 42
8. Construction et publication de l'image Docker 42
9. Déploiement sur Kubernetes 42

10. Vérification du fonctionnement de l'application 43
11. Nettoyage des ressources 43
Conclusion 43
Chapitre 6 : Validation, résultats et analyse critique 44
Introduction 45
1. Résultats obtenus 45
2. Forces du projet 47
3. Limites constatées 47
4. Axes d'amélioration 47
Conclusion 48
Conclusion générale 49

Introduction générale

Dans un contexte où les systèmes informatiques deviennent de plus en plus complexes et distribués, les méthodes traditionnelles de développement et de déploiement montrent leurs limites en termes de rapidité, de fiabilité et de collaboration entre les équipes. L'approche DevOps s'est imposée comme une réponse à ces défis en favorisant l'automatisation, l'intégration continue et la gestion efficace des infrastructures.

Le présent projet s'inscrit dans cette logique en proposant la conception et la mise en œuvre d'une chaîne DevOps complète, couvrant l'ensemble du cycle de vie d'une application, depuis son développement jusqu'à son déploiement sur une infrastructure cloud. L'objectif est de démontrer concrètement comment différentes technologies peuvent être combinées pour automatiser les processus et garantir une meilleure qualité de service.

Pour cela, une application web légère développée en Python avec Flask a été utilisée comme support. Cette application a été conteneurisée à l'aide de Docker, puis intégrée dans un pipeline d'intégration continue basé sur GitHub Actions afin d'automatiser les tests et la construction de l'image. Par la suite, l'infrastructure nécessaire a été provisionnée sur AWS en utilisant Terraform, permettant ainsi une gestion déclarative et reproductible des ressources. La configuration des machines et la mise en place du cluster Kubernetes ont été réalisées via Ansible, assurant une automatisation complète de l'environnement.

Enfin, l'application a été déployée sur un cluster Kubernetes (K3s), permettant de gérer efficacement l'exécution des conteneurs, leur disponibilité et leur exposition. Ce projet met ainsi en évidence l'importance de l'automatisation, de la conteneurisation et de l'orchestration dans la mise en place de solutions modernes et évolutives.

CHAPITRE 1 : CONTEXTE GÉNÉRAL

Introduction

Ce chapitre présente le contexte général dans lequel s'inscrit le projet ainsi que les principaux enjeux liés à la mise en place d'une infrastructure DevOps. Il met en évidence les limites des approches traditionnelles de déploiement des applications et introduit les concepts fondamentaux permettant d'y remédier. Dans ce cadre, l'approche DevOps apparaît comme une solution pertinente pour automatiser et fiabiliser le cycle de vie des applications. Ce chapitre vise également à formuler la problématique du projet et à définir les objectifs à atteindre.

1. Contexte général du projet

Ce projet a été réalisé dans le cadre du Master Data Science, Big Data et Intelligence Artificielle (DSBD & IA) au sein de la Faculté des Sciences Ben M'Sick, relevant de l'Université Hassan II de Casablanca. Il s'inscrit dans une démarche pédagogique visant à mettre en pratique les concepts fondamentaux du DevOps à travers la réalisation d'un projet concret et complet.

L'objectif est de simuler un environnement professionnel réel en mettant en place une chaîne DevOps de bout en bout, capable de gérer l'ensemble du cycle de vie d'une application. Cette chaîne comprend plusieurs couches technologiques interdépendantes, allant du provisionnement de l'infrastructure jusqu'au déploiement automatisé de l'application.

L'architecture mise en place repose sur les éléments suivants :

- Une infrastructure cloud hébergée sur Amazon Web Services (AWS), composée de machines virtuelles (instances EC2)
- L'utilisation de Terraform pour le provisionnement automatique de l'infrastructure (Infrastructure as Code)
- L'utilisation d'Ansible pour la configuration des serveurs et l'installation des composants nécessaires
- La conteneurisation de l'application à l'aide de Docker
- L'orchestration des conteneurs via Kubernetes
- La mise en place d'un pipeline CI/CD pour automatiser les phases de test, de build et de déploiement

Cette approche permet de garantir une infrastructure reproductible, automatisée et conforme aux bonnes pratiques de l'ingénierie logicielle moderne.

2. Problématique

Le déploiement des applications dans les environnements cloud modernes reste souvent basé sur des procédures manuelles, ce qui engendre des erreurs, un manque de fiabilité et une forte dépendance

à l'intervention humaine. Cette approche ne permet pas d'assurer des déploiements rapides et reproductibles.

L'absence de standardisation entre les environnements de développement, de test et de production provoque des incohérences dans l'exécution des applications, rendant leur validation plus complexe.

De plus, le manque d'automatisation du cycle de vie de l'application limite la mise en place de processus fiables d'intégration et de déploiement continus. Cela ralentit la livraison des applications et augmente les risques d'échec en production.

Enfin, la gestion non automatisée de l'infrastructure et l'absence d'orchestration réduisent la capacité à assurer la scalabilité, la résilience et la reproductibilité des systèmes.

Dans ce contexte, le projet vise à mettre en place une infrastructure DevOps permettant d'automatiser le déploiement des applications, tout en garantissant la cohérence des environnements et la fiabilité du processus.

3. Objectifs du projet

a. Objectif général

L'objectif principal de ce projet est de concevoir et de mettre en œuvre une infrastructure DevOps complète permettant d'automatiser le cycle de vie d'une application, depuis son développement jusqu'à son déploiement sur une plateforme cloud.

Cette infrastructure doit être fiable, reproductible et conforme aux bonnes pratiques DevOps, tout en assurant une automatisation maximale des différentes étapes du processus.

b. Objectifs spécifiques

Afin d'atteindre l'objectif général, les objectifs spécifiques suivants ont été définis :

- Développer une application fonctionnelle intégrant des tests unitaires afin de valider son bon fonctionnement.
- Conteneuriser l'application à l'aide de Docker afin de garantir sa portabilité et sa reproductibilité.
- Automatiser le provisionnement de l'infrastructure cloud en utilisant Terraform.
- Configurer automatiquement les machines à l'aide d'Ansible.

- Déployer l'application sur un cluster Kubernetes afin d'assurer son orchestration et sa gestion.
- Mettre en place un pipeline CI/CD permettant d'automatiser les étapes de test, de build et de déploiement.
- Documenter l'ensemble des étapes afin de garantir la reproductibilité de l'infrastructure et faciliter sa réutilisation.

Conclusion

Ce chapitre a permis de présenter le contexte général du projet ainsi que les principaux défis liés au déploiement des applications dans des environnements modernes. Il a également mis en évidence la nécessité d'adopter une approche DevOps pour répondre à ces enjeux. La problématique a été clairement définie, mettant l'accent sur l'automatisation et la fiabilité des processus. Enfin, les objectifs du projet ont été établis afin de guider la réalisation des différentes étapes. Ces éléments constituent une base essentielle pour la suite du rapport.

CHAPITRE 2 : Architecture du Projet

Introduction

Ce chapitre présente l'architecture globale du projet DevOps réalisé. Il décrit les différentes composantes du système ainsi que les interactions entre elles, depuis le développement de l'application jusqu'à son déploiement sur Kubernetes.
L'objectif est de fournir une vision claire et structurée de l'ensemble de la solution mise en place.

1. Vue globale de l'architecture

L'architecture du projet repose sur une approche DevOps complète intégrant plusieurs couches complémentaires :

- couche développement (application Flask)
- couche intégration continue (GitHub Actions)
- couche conteneurisation (Docker)
- couche infrastructure (Terraform / AWS)
- couche configuration (Ansible)
- couche orchestration (Kubernetes K3s)

Le flux global commence par un push du code sur GitHub, déclenchant le pipeline CI/CD, puis se poursuit par la construction de l'image Docker, la mise en place de l'infrastructure et enfin le déploiement de l'application sur le cluster Kubernetes.

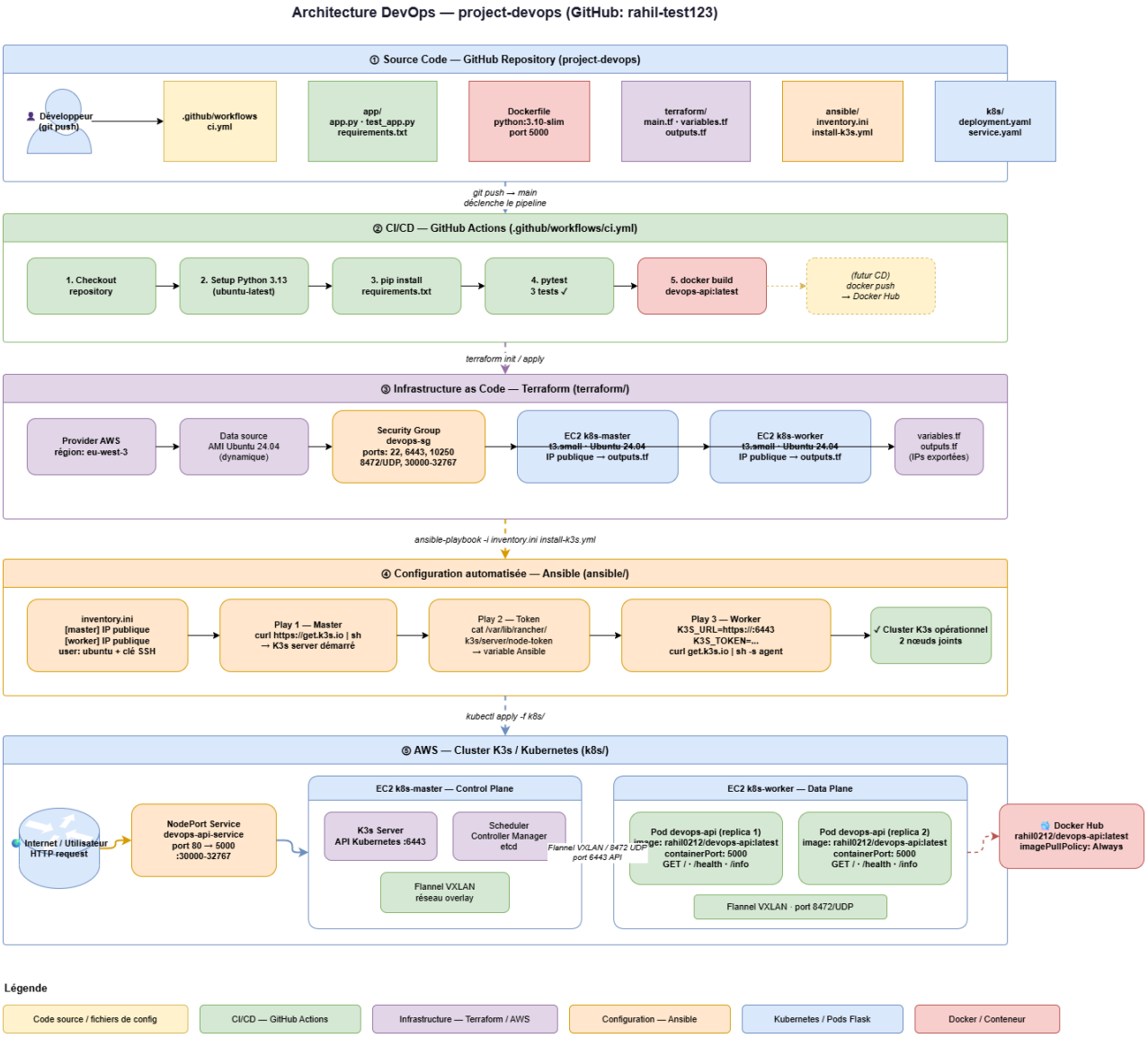

Figure 1-artchitecture du projet

2. Description des différentes couches

a. Couche Application

Cette couche correspond au code source de l'application développée avec Flask. Elle contient les endpoints API, les tests unitaires ainsi que les dépendances nécessaires à son exécution.

L'application constitue le cœur fonctionnel du projet et sert de base pour l'ensemble du processus DevOps.

b. Couche CI/CD (GitHub Actions)

La couche CI/CD automatise la validation et le déploiement du code.
À chaque push sur la branche principale, un workflow GitHub Actions est déclenché.
Ce pipeline effectue :

- l'installation de l'environnement Python
- l'installation des dépendances
- l'exécution des tests unitaires
- la construction de l'image Docker
- la publication de l'image sur Docker Hub
- le déploiement automatique sur un cluster Kubernetes (K3s)

Cette chaîne garantit que le code est non seulement fonctionnel, mais également automatiquement déployé en production dans un environnement conteneurisé, sans intervention manuelle.

c. Couche Conteneurisation (Docker)

L'application est encapsulée dans une image Docker basée sur python:3.10-slim.

Cette couche permet :

- d'uniformiser l'environnement d'exécution
- de faciliter le déploiement
- d'assurer la portabilité de l'application

L'image produite est utilisée ensuite par Kubernetes pour lancer les conteneurs.

d. Couche Infrastructure (Terraform / AWS)

Terraform est utilisé pour provisionner automatiquement l'infrastructure cloud sur AWS.

Les ressources créées sont :

- un VPC et un réseau
- un Security Group
- deux instances EC2 (master et worker)

Cette approche garantit une infrastructure reproductible et versionnée.

e. Couche Configuration (Ansible)

Ansible intervient après la création des machines pour configurer automatiquement les nœuds.

Les actions principales sont :

- installation de K3s sur le master
- récupération du token
- ajout du worker au cluster

Cette étape permet d'obtenir un cluster Kubernetes fonctionnel sans configuration manuelle.

f. Couche Orchestration (Kubernetes K3s)

Kubernetes est utilisé pour déployer et gérer l'application conteneurisée.

Les composants principaux sont :

- un Deployment (gestion des pods)
- un Service NodePort (exposition de l'application)

Le cluster assure :

- la disponibilité de l'application
- la gestion des conteneurs
- la communication réseau entre les composants

3. Flux de fonctionnement global

Le fonctionnement global du système peut être résumé comme suit :

- Le développeur pousse le code sur GitHub
- GitHub Actions déclenche le pipeline CI/CD
- Les tests unitaires sont exécutés
- L'image Docker est construite
- L'image est publiée sur Docker Hub
- Terraform est initialisé et validé
- Les fichiers Kubernetes sont transférés vers le serveur distant
- Kubernetes déploie l'application via les manifests
- Le déploiement est redémarré pour appliquer les modifications
- L'utilisateur accède à l'application via un service NodePort

Ce flux illustre l'intégration complète des outils DevOps dans un cycle automatisé, couvrant à la fois l'intégration continue et le déploiement continu.

Conclusion

L'architecture mise en place repose sur une intégration cohérente de plusieurs outils DevOps couvrant l'ensemble du cycle de vie de l'application.
Elle permet d'automatiser les étapes clés, depuis le développement jusqu'au déploiement, tout en assurant la reproductibilité et la fiabilité de l'infrastructure.

CHAPITRE 3 : Outils et Technologies Utilisés

Introduction

Ce chapitre présente les principaux outils et technologies utilisés dans le cadre de ce projet. Chaque outil a été sélectionné en fonction de son rôle dans l'architecture DevOps et de sa capacité à répondre aux exigences d'automatisation, de reproductibilité et de déploiement continu. L'ensemble de ces technologies permet de construire une infrastructure cohérente couvrant toutes les étapes du cycle de vie de l'application.

1. Tableau récapitulatif des outils / technologies

Outil / Technologie	Catégorie	Rôle dans le projet
Terraform	Infrastructure as Code	Provisionnement des ressources AWS (EC2, réseau, sécurité)
Ansible	Gestion de configuration	Installation de Docker, Kubernetes et configuration des nœuds
Docker	Conteneurisation	Création de l'image de l'application
Docker Hub	Registry	Stockage et distribution de l'image Docker
Kubernetes (K3s)	Orchestration	Déploiement et gestion des conteneurs
Flannel CNI	Réseau Kubernetes	Communication entre les pods
CI/CD (GitHub Actions)	Intégration continue	Automatisation des tests, build et déploiement
Runner CI/CD	Exécuteur	Exécution des jobs du pipeline
AWS EC2	Cloud	Hébergement des machines (master et worker)
AWS VPC / Security Group	Réseau cloud	Gestion des accès et isolation réseau
Python / Flask	Développement	Implémentation de l'application web
pytest	Tests unitaires	Validation du fonctionnement de l'application
SSH / ProxyJump	Accès sécurisé	Connexion au worker via le master
Linux (Amazon Linux / Ubuntu)	Système d'exploitation	Environnement d'exécution des machines

Tableau 1-1. Tableau récapitulatif des outils / technologies

2. Justification des choix technologiques

- **AWS(FreeTier)**
  Le choix d'AWS repose sur la disponibilité de ressources cloud accessibles via le Free Tier, permettant de déployer une infrastructure réelle sans coût supplémentaire. Les instances EC2 offrent un environnement flexible pour héberger les nœuds du cluster Kubernetes, tout en respectant les contraintes du projet.

- **Terraform**
  Terraform a été utilisé pour automatiser le provisionnement de l'infrastructure. Son approche déclarative permet de définir les ressources cloud sous forme de code, garantissant ainsi la reproductibilité et la cohérence de l'environnement. La commande terraform plan permet de vérifier les modifications avant leur application.

- **Ansible**
  Ansible est utilisé pour automatiser la configuration des machines après leur création. Son fonctionnement sans agent (agentless) et son utilisation de fichiers YAML facilitent la mise en place des configurations. Il permet d'installer Docker, Kubernetes et de configurer le cluster de manière homogène sur tous les nœuds.

- **Docker**
  Docker permet de conteneuriser l'application, assurant ainsi la portabilité et la cohérence de son exécution. L'image Docker est construite à partir d'un environnement Python optimisé, ce qui garantit un déploiement identique sur tous les environnements.

- **DockerHub**
  Docker Hub est utilisé comme registre pour stocker et distribuer les images Docker. Il permet au cluster Kubernetes de récupérer automatiquement l'image de l'application lors du déploiement.

- **Kubernetes(kubeadm)**
  Kubernetes est utilisé via K3s, une distribution légère de Kubernetes adaptée aux environnements simples et aux ressources limitées. Il permet de déployer rapidement un cluster tout en conservant les fonctionnalités essentielles de Kubernetes.

- **FlannelCNI**
  Flannel est utilisé comme solution réseau pour Kubernetes. Il permet la communication entre les pods sur différents nœuds du cluster, assurant ainsi le bon fonctionnement des services distribués.

- **GitHubActions(CI/CD)**
  GitHub Actions est utilisé pour automatiser le pipeline CI/CD. Il permet d'exécuter automatiquement les tests, de construire l'image Docker et de la publier. Chaque modification du code déclenche le workflow, garantissant un processus de déploiement continu et fiable.

- **Python/Flask**
  Flask a été utilisé pour développer l'application web. Il s'agit d'un framework léger, adapté à la création d'API REST et facile à intégrer dans une architecture conteneurisée.
- **pytest**
  pytest permet de réaliser des tests unitaires sur l'application. Il est intégré dans le pipeline CI afin de valider automatiquement le bon fonctionnement du code avant le déploiement.
- **SSH/ProxyJump**
  L'utilisation de SSH avec ProxyJump permet de sécuriser l'accès au nœud worker en passant par le nœud master. Cela évite l'exposition directe du worker à Internet et renforce la sécurité de l'infrastructure.

Conclusion

Les outils utilisés dans ce projet forment un ensemble cohérent couvrant toutes les étapes du cycle DevOps, depuis le provisionnement de l'infrastructure jusqu'au déploiement de l'application. Chaque technologie joue un rôle spécifique et complémentaire, permettant d'automatiser les processus et de garantir la fiabilité du système. L'intégration de ces outils permet de mettre en place une infrastructure moderne, reproductible et conforme aux bonnes pratiques DevOps.

Chapitre 4 : Conception et Mise en œuvre de la solution

Introduction

Ce chapitre présente la mise en œuvre technique de la solution DevOps adoptée dans le cadre de ce projet. Il décrit l'ensemble des étapes réalisées, depuis le développement de l'application jusqu'à son déploiement sur une infrastructure cloud basée sur Kubernetes. L'objectif est de montrer comment les différents outils et technologies notamment Docker, GitHub Actions, Terraform, Ansible et Kubernetes ont été intégrés afin de construire une chaîne complète d'automatisation. Cette démarche permet d'illustrer concrètement les principes du DevOps à travers une architecture cohérente et reproductible.

1. Développement de l'application et conteneurisation

a. Présentation fonctionnelle de l'application

L'application choisie est une application web complète développée en Python avec Flask, combinant à la fois une API REST et une interface utilisateur simple. Ce choix reste cohérent avec l'objectif du projet DevOps : disposer d'un service testable, conteneurisable et déployable, tout en intégrant des fonctionnalités plus riches pour se rapprocher d'un cas réel.

Contrairement à la première version minimaliste, l'application actuelle propose une structure plus avancée incluant :

- une interface web basée sur des templates HTML,
- une API REST permettant de manipuler des données musicales (playlists, albums, chansons),
- un système de recherche dynamique,
- et un mécanisme simulant un lecteur audio.

Le fichier principal app.py expose plusieurs routes HTTP organisées entre routes web et endpoints API :

Endpoint	Méthode	Rôle	Réponse attendue
/	GET	Affiche la page principale de l'application.	Page HTML (interface utilisateur)
/health	GET	Point de santé pour superviser le service.	{"status": "OK"}
/info	GET	Retourne des métadonnées sur le projet.	{"project": "DevOps Cloud Project", "version": "2.0"}
/api/playlists	GET	Retourne la liste des playlists.	JSON contenant les playlists

/api/albums	GET	Retourne la liste des albums.	JSON contenant les albums
/api/songs	GET	Retourne la liste des chansons.	JSON contenant les chansons
/api/search?q=...	GET	Permet de rechercher des éléments.	Résultats filtrés en JSON
/api/player/{category}/{id}	GET	Retourne les détails pour le lecteur audio.	JSON détaillé d'un élément

Tableau 2- Description des endpoints de l'API Flask

Ce choix fonctionnel présente plusieurs avantages pédagogiques. Premièrement, l'application reste suffisamment simple pour être testée facilement, tout en introduisant une logique métier basique (gestion et recherche de contenu). Deuxièmement, elle permet de valider le bon fonctionnement du service aussi bien en local, dans un conteneur Docker que dans un cluster Kubernetes. Troisièmement, la présence d'un endpoint /health s'intègre naturellement dans une logique de monitoring et de sondes de santé. Enfin, l'ajout d'une interface utilisateur et d'une API plus riche permet de simuler un cas d'usage réaliste, ce qui renforce la pertinence du projet dans un contexte DevOps.

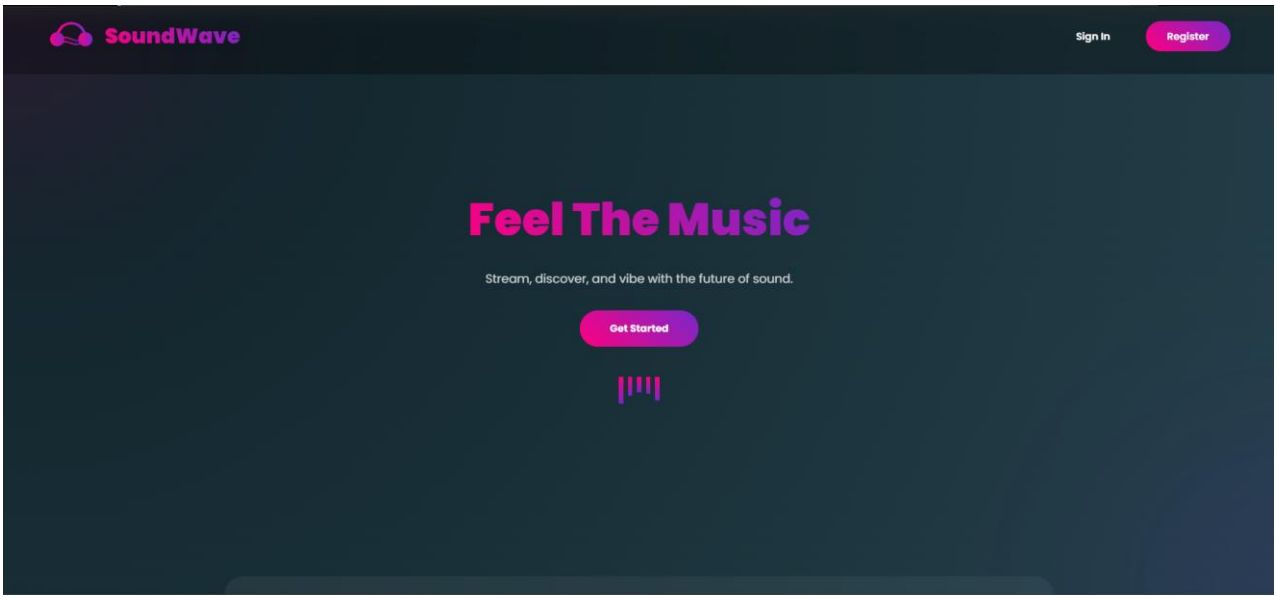

Figure 2-Page d'accueil de l'application web

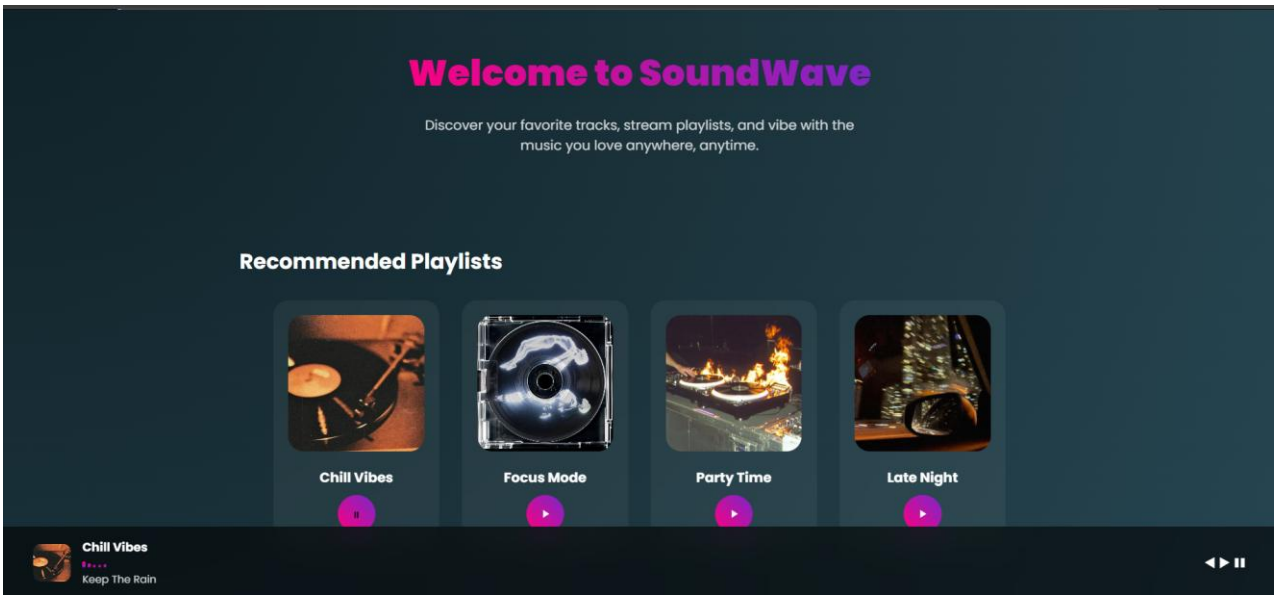

Figure 3-Interface de navigation des playlists et contenus musicaux

b. Tests unitaires

Afin de garantir la qualité du code, des tests unitaires ont été implémentés à l'aide de pytest. Ces tests permettent de vérifier le bon fonctionnement des endpoints de l'application.

Les tests couvrent notamment :

- la disponibilité de l'API
- la validité des réponses retournées
- le bon fonctionnement du endpoint /health.

L'exécution des tests est intégrée dans le pipeline CI, ce qui permet de valider automatiquement le code à chaque modification.

c. Conteneurisation avec Docker

La conteneurisation de l'application est réalisée à l'aide d'un Dockerfile basé sur l'image python:3.10-slim. Le conteneur définit /app comme répertoire de travail, copie le fichier requirements.txt, installe les dépendances avec pip, puis copie le code source de l'application. Le port 5000 est exposé et l'application est lancée avec la commande python app.py.

Cette organisation respecte une bonne pratique importante. La copie du fichier des dépendances avant celle du code permet d'exploiter le cache Docker. Tant que les dépendances ne changent pas, les couches précédentes sont réutilisées, ce qui réduit le temps de construction de l'image.

La conteneurisation répond à plusieurs objectifs :

- standardiser l'environnement d'exécution ;
- embarquer toutes les dépendances de l'application ;
- faciliter la distribution de l'image via un registre ;
- assurer la cohérence entre les environnements de développement et de production.

Dans les manifests Kubernetes, l'image utilisée est rahil0212/devops-api:latest, ce qui implique que l'image doit être construite et publiée dans un registre avant le déploiement

```
C:\Users\hp\devops-project\app>docker ps
CONTAINER ID   IMAGE        COMMAND           CREATED          STATUS          PORTS                                          NAMES
1fcf73ba0cb5   devops-api   "python app.py"   22 seconds ago   Up 20 seconds   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   devops-api-container
```

Figure 4-Vérification du conteneur Docker en cours d'exécution avec la commande

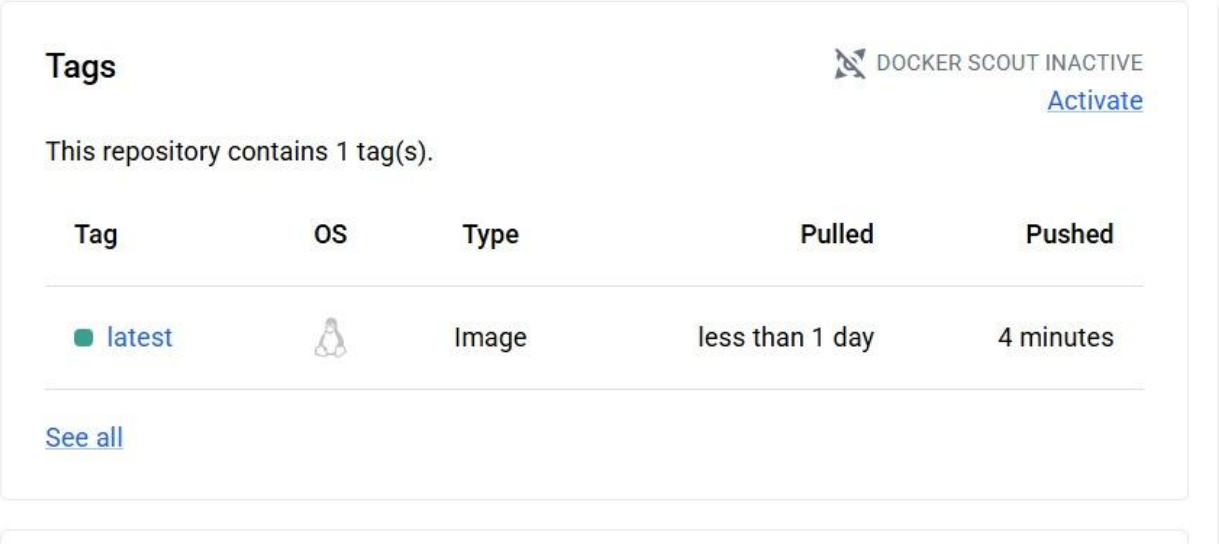

Figure 5-Image Docker publiée sur Docker Hub

2. Intégration continue (CI/CD)

a. Mise en place du pipeline

Le dépôt contient un workflow GitHub Actions nommé DevOps CI/CD Pipeline. Ce workflow se déclenche sur chaque push vers la branche main. Il définit un job build-test-deploy exécuté sur ubuntu-latest et composé de plusieurs étapes : récupération du dépôt (actions/checkout), installation de Python 3.10, installation des dépendances du projet puis exécution des tests avec pytest.

À la suite de ces étapes, le workflow construit une image Docker de l'application, puis la publie sur Docker Hub. Enfin, le pipeline réalise automatiquement le déploiement de l'application sur un cluster Kubernetes (K3s) en transférant les fichiers de configuration et en appliquant les manifests via une connexion SSH.

D'un point de vue méthodologique, ce pipeline remplit le rôle d'une chaîne complète CI/CD : il automatise à la fois la validation du code et son déploiement. Cela signifie que toute modification poussée sur la branche principale est non seulement testée et validée, mais également déployée automatiquement dans un environnement de production. Ce processus permet de réduire les erreurs humaines, d'accélérer les mises à jour et d'assurer une meilleure fiabilité du système.

Étape du workflow	Objectif	Commentaire
Checkout repository	Récupérer la dernière version du code	Nécessaire pour rendre le dépôt disponible sur le runner.
Set up Python	Préparer l'environnement d'exécution	Le workflow utilise Python 3.10 pour exécuter les tests.
Install dependencies	Installer Flask et pytest	Les dépendances sont installées à partir de requirements.txt.
Run tests	Valider le code applicatif	Les tests unitaires sont exécutés via pytest.
Build Docker image	Construire l'image de l'application	L'image Docker est générée à partir du Dockerfile.
Push Docker image	Publier l'image	L'image est envoyée vers Docker Hub pour être utilisée lors du déploiement.
Terraform Init / Validate / Plan	Vérifier la configuration d'infrastructure	Permet de valider les fichiers Terraform avant déploiement.
Setup SSH	Préparer la connexion sécurisée	La clé privée est utilisée pour accéder au serveur distant.
Copy Kubernetes manifests	Transférer les fichiers de configuration	Les fichiers Kubernetes sont copiés vers le nœud master.
Deploy to Kubernetes	Déployer l'application	Les manifests sont appliqués avec kubectl et le déploiement est mis à jour.
Verify deployment	Vérifier l'état du déploiement	Permet de s'assurer que les pods et services sont actifs.

Tableau 3-Description des étapes du pipeline CI/CD

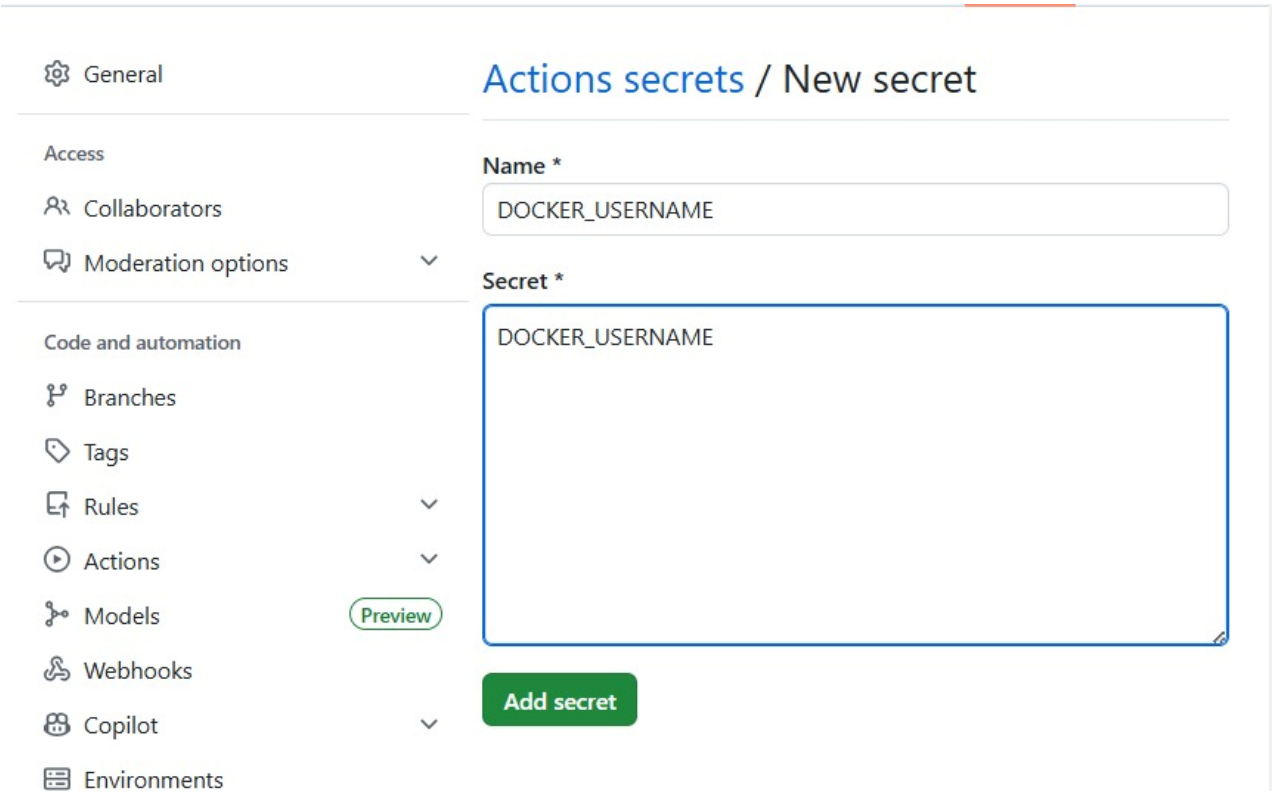

Figure 6-Configuration des secrets GitHub Actions pour l'authentification Docker Hub

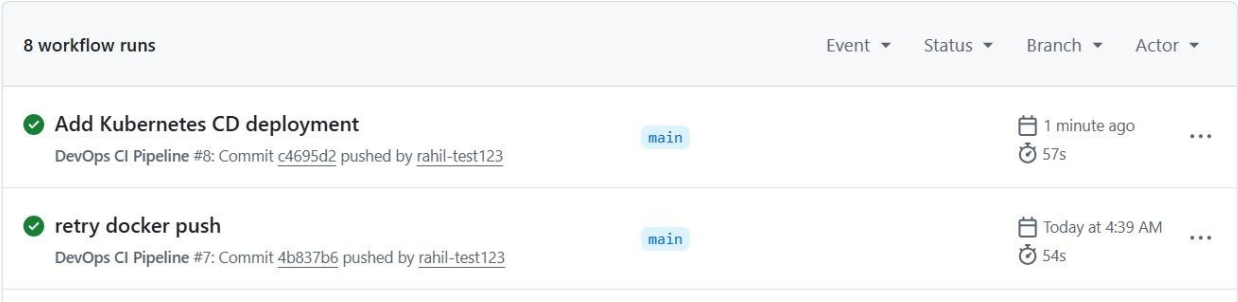

Figure 7-Exécution réussie du pipeline CI/CD dans GitHub Actions

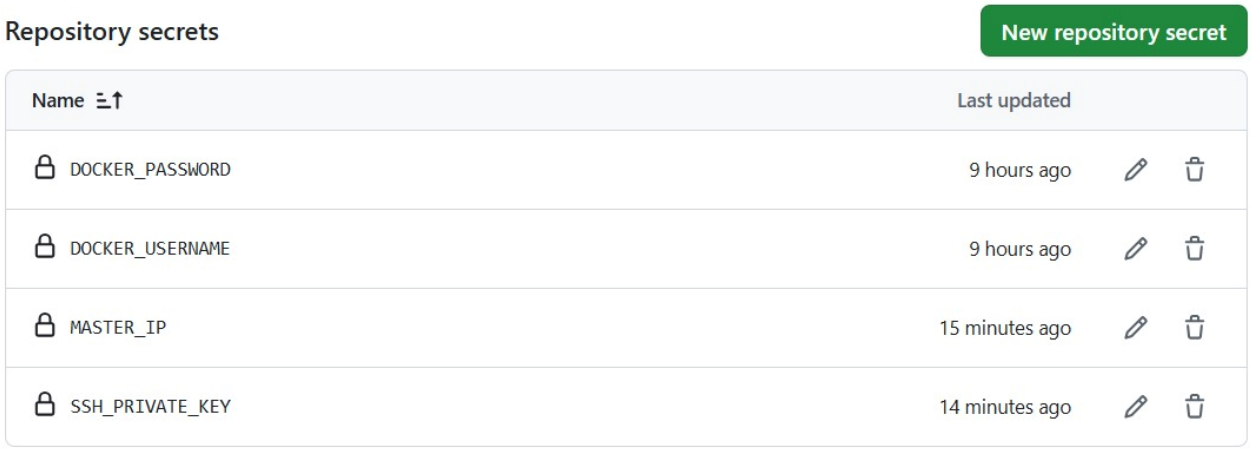

Figure 8-Configuration des secrets du dépôt dans GitHub Actions

d. Analyse du pipeline

Le pipeline mis en place correspond à une chaîne CI/CD fonctionnelle.
Il automatise les tests, la construction de l'image Docker ainsi que sa publication sur Docker Hub.

Le pipeline assure également le déploiement de l'application sur Kubernetes en appliquant automatiquement les manifests via une connexion SSH au nœud master.

Cette automatisation permet de réduire les interventions manuelles et d'assurer une mise à jour rapide de l'application après chaque modification du code.

Toutefois, certaines améliorations peuvent être envisagées, notamment l'ajout d'une étape de planification Terraform complète ou l'optimisation de la gestion des versions des images Docker.

3. Provisionnement de l'infrastructure avec Terraform

La mise en place de l'infrastructure cloud est assurée par Terraform. Le fichier `main.tf` déclare le provider AWS, sélectionne une image Ubuntu récente grâce à la source de données `aws_ami`, crée un groupe de sécurité nommé `devops-sg`, puis provisionne deux instances EC2 : une pour le nœud master et une pour le nœud worker.

Le choix de Terraform est particulièrement pertinent dans un projet DevOps, car il introduit le principe d'Infrastructure as Code. L'infrastructure n'est plus créée manuellement via l'interface graphique du fournisseur cloud ; elle est décrite sous forme déclarative dans des fichiers versionnables, relisibles, auditables et reproductibles. Cette approche réduit les erreurs humaines, facilite la reconstruction de l'environnement et permet un meilleur suivi des changements.

```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

master_private_ip = "172.31.42.82"
master_public_ip = "52.47.182.139"
worker_private_ip = "172.31.46.232"
worker_public_ip = "13.39.148.25"

PC@DESKTOP-LST7EAV MINGW64 ~/devops-project/terraform
$
```

Figure 9-Résultat de la commande Terraform affichant les adresses IP publiques et privées des instances EC2

a. Ressources définies

Dans l'implémentation étudiée, Terraform crée les éléments suivants :

- un provider AWS configuré sur la région `eu-west-3` ;
- une AMI Ubuntu 24.04 récente récupérée dynamiquement ;
- un security group autorisant SSH, l'API Kubernetes, le port kubelet, le trafic VXLAN de Flannel, la plage NodePort et le trafic interne entre nœuds ;
- deux instances EC2 de type `t3.small`, nommées `k8s-master` et `k8s-worker`.

Port / flux	Usage	Évaluation
22/TCP	Administration SSH	Nécessaire pour l'accès distant et Ansible.
6443/TCP	API Kubernetes / K3s server	Indispensable au fonctionnement du cluster.
10250/TCP	Kubelet	Utilisé pour la communication de certains composants Kubernetes.
8472/UDP	Flannel VXLAN	Nécessaire au réseau overlay inter-pods de K3s/Flannel.
30000–32767/TCP	Services NodePort	Permet l'exposition d'un service applicatif vers l'extérieur.
Trafic interne entre nœuds	Communication master/worker	Facilite la connectivité au sein du cluster.

Tableau 4-Ports et flux reseau utilises dans le cluster Kubernetes

Le fichier `outputs.tf` complète cette logique en exportant les IP publiques et privées des deux instances. Cette pratique est importante, car elle permet d'exploiter automatiquement les valeurs produites par Terraform dans d'autres étapes du flux de déploiement, par exemple pour alimenter un inventaire Ansible ou pour préparer un fichier de configuration.

```
Outputs:

master_public_ip = "15.188.51.174"
worker_public_ip = "52.47.130.170"

C:\Users\PC\devops-project\terraform>
```

Figure 10-Affichage des adresses IP publiques des instances EC2 générées par Terraform

b. Analyse critique de la configuration Terraform

L'analyse détaillée du code révèle cependant un point d'amélioration significatif : le fichier `variables.tf` déclare des variables pour la région, le nom de la clé et les types d'instances, mais ces variables ne sont pas réellement consommées dans `main.tf`. Les valeurs sont en effet codées en dur dans la définition du provider et des instances. La présence des variables montre une intention correcte d'abstraction, mais le fichier `main.tf` devrait utiliser `var.aws_region`, `var.key_name`, `var.master_instance_type` et

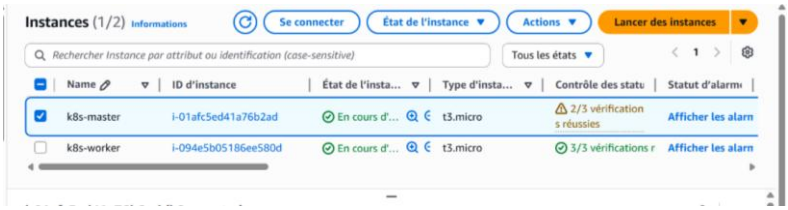

Figure 11-Instances EC2 créées sur AWS pour le cluster Kubernetes (master et worker)

`var.worker_instance_type` pour rendre l'infrastructure vraiment paramétrable.

De même, le security group autorise l'accès depuis `0.0.0.0/0` sur plusieurs ports. Cette ouverture large se justifie pour une démonstration académique et pour accélérer la mise en service, mais elle ne serait pas acceptable telle quelle dans un environnement de production. Une version durcie limiterait l'accès SSH à une adresse IP d'administration connue, restreindrait l'API Kubernetes et utiliserait éventuellement un VPN ou un bastion.

4. Configuration automatisée avec Ansible et mise en place du cluster K3s

Après le provisionnement des machines, la seconde phase du projet consiste à transformer deux serveurs Ubuntu nus en un cluster Kubernetes fonctionnel. Cette étape est prise en charge par Ansible à travers un inventaire simple et un playbook `install-k3s.yml`. Le choix d'Ansible est adapté à ce scénario : l'outil permet d'exécuter des opérations à distance via SSH, sans agent supplémentaire, tout en décrivant les étapes de manière lisible et répétable.

a. Inventaire Ansible

Le fichier `inventory.ini` déclare deux groupes d'hôtes, `master` et `worker`, ainsi que des variables communes : l'utilisateur `ubuntu` et le chemin vers la clé privée SSH. Dans la version fournie, les adresses IP publiques sont inscrites directement dans le fichier d'inventaire. Cette solution fonctionne pour une démonstration, mais elle introduit un couplage fort entre l'inventaire et l'exécution courante de Terraform.

Une amélioration naturelle consisterait à générer automatiquement l'inventaire à partir des sorties Terraform, voire à utiliser un inventaire dynamique AWS. Cela éviterait de modifier manuellement les IP à chaque destruction/recréation de l'environnement.

```
ubuntu@ip-172-31-42-82:~/ansible$ ansible all -i inventory.ini -m ping
worker | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
master | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
```

Figure 12-Vérification de la connectivité des nœuds avec Ansible (commande ping)

b. Installation du master K3s

Le playbook commence par installer K3s en mode serveur sur l'hôte `master` au moyen d'un script shell récupéré depuis `https://get.k3s.io`. Une fois l'installation terminée, le nœud maître initialise le cluster et génère un token d'authentification que les agents doivent fournir pour rejoindre ce cluster.

c. Récupération du token et jonction du worker

Dans un second temps, le playbook lit le fichier `/var/lib/rancher/k3s/server/node-token`, enregistre son contenu dans une variable Ansible puis l'exploite dans la phase de déploiement du worker. Le nœud `worker` installe K3s en mode agent et utilise ce token pour rejoindre le serveur maître au travers de la variable `K3S_URL` pointant vers l'adresse privée du master sur le port 6443.

Cette séquence illustre bien le rôle d'Ansible dans le projet : sans intervention manuelle sur chaque machine, le cluster est initialisé, sécurisé par token et constitué en une seule exécution. L'automatisation de cette étape représente un gain de temps notable et garantit la reproductibilité du cluster.

```
ubuntu@ip-172-31-42-82:~$ sudo k3s kubectl get nodes
NAME                STATUS   ROLES           AGE    VERSION
ip-172-31-42-82     Ready    control-plane   2d2h   v1.34.5+k3s1
ip-172-31-46-232    Ready    <none>          13s    v1.34.5+k3s1
ubuntu@ip-172-31-42-82:~$
```

```
ubuntu@ip-172-31-42-82:~/ansible$ ansible-playbook -i inventory.ini install-k3s.yml

PLAY [Install K3s on master] ***************************************************

TASK [Gathering Facts] *********************************************************
ok: [master]

TASK [Install K3s server] ******************************************************
changed: [master]

PLAY [Get K3s token from master] ***********************************************

TASK [Gathering Facts] *********************************************************
ok: [master]

TASK [Read node token] *********************************************************
changed: [master]

TASK [Set token fact] **********************************************************
ok: [master]

PLAY [Install K3s on worker] ***************************************************

TASK [Gathering Facts] *********************************************************
ok: [worker]

TASK [Install K3s agent] *******************************************************
changed: [worker]

PLAY RECAP *********************************************************************
master                     : ok=5    changed=2    unreachable=0    failed=0    skipp
ed=0    rescued=0    ignored=0
worker                     : ok=2    changed=1    unreachable=0    failed=0    skipp
ed=0    rescued=0    ignored=0

ubuntu@ip-172-31-42-82:~/ansible$
```

Figure 14-Exécution du playbook Ansible pour l'installation du cluster K3s (master et worker)

```
ubuntu@ip-172-31-42-82:~/ansible$ sudo k3s kubectl get nodes -o wide
NAME                STATUS   ROLES           AGE    VERSION        INTERNAL-IP     EX
TERNAL-IP   OS-IMAGE             KERNEL-VERSION    CONTAINER-RUNTIME
ip-172-31-42-82     Ready    control-plane   2d3h   v1.34.5+k3s1   172.31.42.82    <n
one>        Ubuntu 24.04.4 LTS   6.17.0-1007-aws   containerd://2.1.5-k3s1
ip-172-31-46-232    Ready    <none>          81m    v1.34.5+k3s1   172.31.46.232   <n
one>        Ubuntu 24.04.4 LTS   6.17.0-1007-aws   containerd://2.1.5-k3s1
ubuntu@ip-172-31-42-82:~/ansible$
```

Figure 13-Vérification détaillée des nœuds du cluster K3s avec leurs adresses IP et leur état

d. Analyse critique du playbook

Le playbook fourni est volontairement compact, ce qui le rend facile à comprendre. Néanmoins, il présente un point d'attention important : l'adresse privée du master est codée en dur dans la ligne `K3S_URL=https://<MASTER_PRIVATE_IP>:6443`. Cette pratique est acceptable pour une démonstration, mais elle réduit la portabilité du playbook. Une version plus robuste utiliserait une variable issue de Terraform ou une variable d'inventaire, ce qui rendrait le déploiement totalement indépendant d'une IP fixe.

Par ailleurs, la configuration pourrait être enrichie par des tâches supplémentaires : installation préalable des paquets utiles, vérifications d'état, gestion idempotente plus poussée, ou copie du fichier kubeconfig vers un emplacement facilitant l'administration. Malgré cela, la version actuelle répond au besoin principal : créer un cluster K3s opérationnel à partir de deux machines fraîchement provisionnées.

e. Emplacement des livrables Ansible

Les explications figurent dans ce chapitre, tandis que l'inventaire Ansible et le playbook `install-k3s.yml` sont reproduits dans l'Annexe D. Pour des raisons de sécurité, la clé privée `devops-key.pem` n'est pas copiée dans le rapport ; seule sa référence dans les fichiers de configuration est mentionnée.

5. Déploiement de l'application sur Kubernetes

Une fois le cluster K3s disponible, l'application peut être déployée à l'aide de manifests Kubernetes. Le projet fournit deux fichiers YAML : `deployment.yaml` et `service.yaml`. Ces fichiers matérialisent la partie déploiement de la chaîne DevOps en traduisant l'application conteneurisée en objets compréhensibles par Kubernetes.

a. Analyse du Deployment

Le manifeste `deployment.yaml` définit une ressource `Deployment` nommée `devops-api`. Le nombre de réplicas est fixé à 2, ce qui signifie que Kubernetes doit maintenir deux pods disponibles pour servir l'application. Cette décision est intéressante même dans un petit projet, car elle illustre immédiatement un avantage majeur de Kubernetes : la capacité à répliquer un service pour améliorer sa disponibilité et répartir la charge.

Le template du pod référence l'image `rahil0212/devops-api:latest` avec `imagePullPolicy: Always`, ce qui force le cluster à reconsulter le registre à chaque redémarrage de pod. Le conteneur expose le port 5000, cohérent avec l'application Flask. Dans une perspective d'industrialisation, ce manifeste pourrait être enrichi par des ressources CPU/Mémoire, des probes de santé, une stratégie de rolling update et éventuellement une variable d'environnement pour le mode d'exécution.

b. Analyse du Service

Le manifeste `service.yaml` définit un service de type `NodePort` nommé `devops-api-service`. Le service sélectionne les pods portant l'étiquette `app: devops-api`, expose le port 80 côté cluster et redirige le trafic vers le port 5000 du conteneur. Le choix de `NodePort` est cohérent avec un cluster simple à deux nœuds sur AWS : il offre un mécanisme d'exposition externe sans nécessiter d'ingress controller ni de load balancer managé.

Le revers de ce choix est que l'utilisateur doit connaître l'adresse d'un nœud ainsi que le port NodePort attribué par Kubernetes. Dans un environnement de production, on privilégierait généralement un service de type `LoadBalancer` ou un Ingress combiné à un contrôleur approprié. Pour une maquette académique, `NodePort` est néanmoins suffisant et permet de visualiser clairement le chemin réseau entre l'extérieur et les pods.

c. Processus type de déploiement

Une procédure de déploiement cohérente sur le master K3s peut être résumée par les commandes suivantes :

- sudo k3s kubectl get nodes -o wide
- sudo k3s kubectl apply -f k8s/deployment.yaml
- sudo k3s kubectl apply -f k8s/service.yaml
- sudo k3s kubectl get pods -o wide

- sudo k3s kubectl get svc

Ces commandes permettent successivement de vérifier l'état du cluster, de créer le déploiement, d'exposer le service puis de contrôler l'état des pods et des services. Lorsque tout est correctement configuré, l'API devient accessible via l'adresse IP d'un nœud et le NodePort alloué.

```
ubuntu@ip-172-31-42-82:~$ kubectl get podsds
kubectl get svc
NAME                          READY   STATUS    RESTARTS   AGE
devops-api-668d6f4dc-bvq97    1/1     Running   0          2m18s
devops-api-668d6f4dc-fbgqr    1/1     Running   0          2m11s
NAME                 TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)
 AGE
devops-api-service   NodePort    10.43.178.38   <none>        80:32539/TCP
 28m
kubernetes           ClusterIP   10.43.0.1      <none>        443/TCP
 2d5h
ubuntu@ip-172-31-42-82:~$
```

Figure 15-Vérification des pods et du service Kubernetes avec la commande kubectl

d. Monitoring optionnel

La version remise ne contient pas de stack de monitoring opérationnelle basée sur Grafana, Loki, Prometheus ou Promtail. Il est donc préférable de l'indiquer explicitement dans le rapport plutôt que d'affirmer une fonctionnalité non réalisée. Cela n'empêche pas de prévoir une section dédiée, car le sujet mentionne le monitoring comme livrable optionnel.

Dans une extension du projet, Prometheus pourrait être utilisé pour collecter les métriques des nœuds et des pods, Grafana pour la visualisation, Loki pour l'agrégation des logs et Promtail comme agent de collecte. Cette évolution viendrait compléter naturellement l'endpoint `/health` déjà présent dans l'application. L'emplacement recommandé pour documenter un monitoring futur est une sous-section du chapitre 8 dans le corps du rapport, avec les manifests ou valeurs Helm en Annexe F.

e. Emplacement des livrables Kubernetes

Les manifests YAML complets de Kubernetes sont fournis dans l'Annexe E. Le présent chapitre explique leur logique et leur rôle dans l'exposition de l'application sur le cluster.

Conclusion

Au terme de ce chapitre, l'ensemble des étapes de mise en œuvre de la solution a été détaillé, mettant en évidence l'intégration cohérente des différentes technologies utilisées. Le projet couvre l'ensemble du cycle DevOps, depuis le développement et les tests jusqu'au déploiement sur une infrastructure cloud automatisée. Les choix techniques réalisés permettent d'assurer une certaine reproductibilité et une bonne organisation de l'environnement. Cette mise en œuvre constitue ainsi une base solide pour l'analyse des résultats et l'évaluation critique présentées dans le chapitre suivant.

Chapitre 5 : Documentation pour la reproduction de l'infrastructure

Introduction

Ce chapitre présente la procédure détaillée permettant de reproduire l'ensemble de l'infrastructure mise en place dans ce projet. Il a pour objectif de guider un utilisateur externe dans le déploiement complet de la solution, depuis la création des ressources cloud jusqu'à la mise en service de l'application sur Kubernetes. Cette démarche s'inscrit dans une logique DevOps où la reproductibilité et l'automatisation constituent des éléments essentiels.

1. Prérequis

Avant de commencer le déploiement, il est nécessaire de disposer des éléments suivants :

- Un compte AWS actif avec les permissions nécessaires pour créer des instances EC2 et des Security Groups.
- Une clé SSH configurée sur AWS (nommée devops-key) ainsi que le fichier privé associé (devops-key.pem) disponible sur la machine locale.
- Terraform installé et accessible en ligne de commande.
- Ansible installé sur la machine d'administration.
- Docker installé pour la construction des images.
- Un client SSH fonctionnel.
- Une connexion Internet permettant le téléchargement des dépendances et des outils.

Il est également nécessaire de sécuriser la clé privée avec la commande suivante :

```
chmod 400 devops-key.pem
```

2. Préparation du projet

Le projet est organisé en plusieurs répertoires correspondant aux différentes composantes de l'architecture :

- app/ : code source de l'application Flask
- k8s/ : manifests Kubernetes
- terraform/ : fichiers de provisionnement de l'infrastructure
- ansible/ : playbooks de configuration du cluster
- keys/ : fichier de clé privée (non versionné en pratique)

Cette organisation permet de structurer clairement les différentes étapes du déploiement.

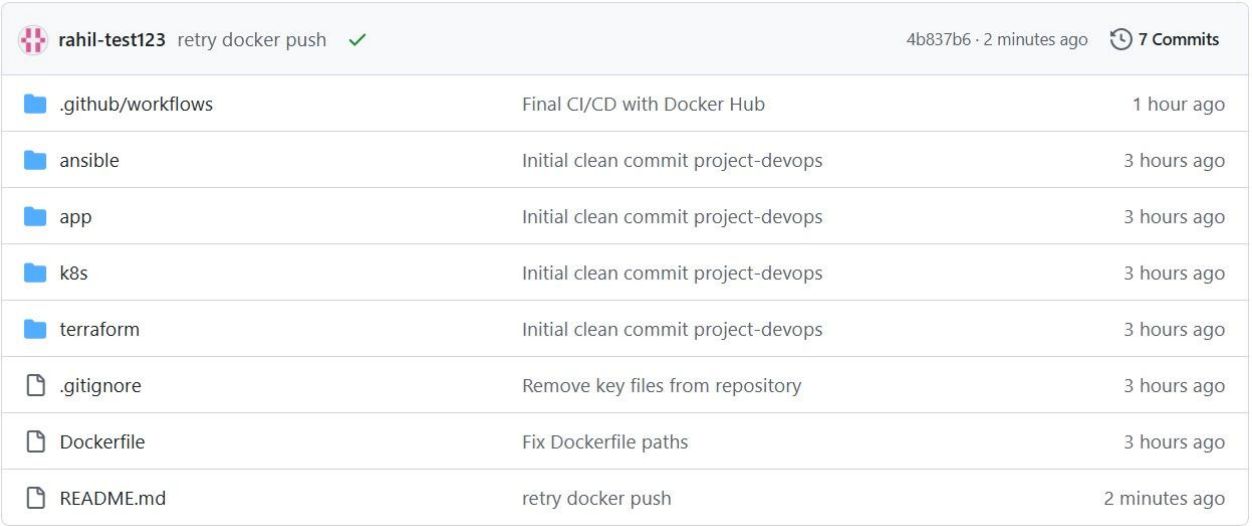

Figure 16-Structure du dépôt GitHub du projet DevOps

3. Provisionnement de l'infrastructure avec Terraform

Le provisionnement de l'infrastructure AWS est réalisé à l'aide de Terraform.

Depuis le dossier terraform/, exécuter les commandes suivantes :

```
terraform init
terraform plan
terraform apply
```

Après validation, Terraform crée automatiquement :

- le Security Group
- les instances EC2 (master et worker)

Une fois l'exécution terminée, les adresses IP des machines sont récupérées à l'aide des commandes suivantes :

```
terraform output master_public_ip
terraform output worker_public_ip
terraform output master_private_ip
terraform output worker_private_ip
```

4. Configuration de l'inventaire Ansible

Le fichier inventory.ini doit être mis à jour avec les adresses IP générées par Terraform.

- Le groupe master contient l'IP publique du nœud master

- Le groupe worker contient l'IP publique du nœud worker
- L'utilisateur distant est ubuntu
- Le chemin vers la clé privée doit être correctement configuré

Le playbook utilise également l'adresse privée du master pour permettre au worker de rejoindre le cluster. Cette valeur doit correspondre à master_private_ip.

5. Installation et configuration du cluster Kubernetes (K3s)

La configuration du cluster est réalisée à l'aide d'Ansible.

Avant d'exécuter le playbook, il est recommandé de vérifier la connectivité :

```
ansible all -i inventory.ini -m ping
```

Ensuite, lancer le playbook :

```
ansible-playbook -i inventory.ini install-k3s.yml
```

Ce playbook réalise les opérations suivantes :

- installation de K3s en mode serveur sur le nœud master
- génération du token du cluster
- installation de K3s en mode agent sur le worker
- jonction automatique du worker au cluster

À la fin de cette étape, le cluster Kubernetes est opérationnel.

6. Vérification du cluster

Se connecter au nœud master :

```
ssh -i ./keys/devops-key.pem ubuntu@<MASTER_PUBLIC_IP>
```

Puis vérifier l'état du cluster :

```
sudo k3s kubectl get nodes -o wide
```

Le résultat attendu est la présence de deux nœuds (master et worker) avec l'état Ready.

7. Test local de l'application

Avant le déploiement, il est recommandé de tester l'application localement :

```
cd app
pip install -r
requirements.txt
pytest
python app.py
```

Cette étape permet de vérifier que l'application fonctionne correctement avant la conteneurisation.

8. Construction et publication de l'image Docker

Construire l'image Docker :

```
docker build -t devops-api
```

Taguer l'image pour le registre :

```
docker tag devops-api rahil0212/devops-api:latest
```

Publier l'image :

```
docker push rahil0212/devops-api:latest
```

Cette étape permet de rendre l'image accessible au cluster Kubernetes.

---

9. Déploiement sur Kubernetes

Depuis le nœud master, appliquer les manifests :

```
sudo k3s kubectl apply -f k8s/deployment.yaml
sudo k3s kubectl apply -f k8s/service.yaml
```

Vérifier l'état des ressources :

```
sudo k3s kubectl get pods
sudo k3s kubectl get svc
```

Le service NodePort permet d'accéder à l'application depuis l'extérieur.

10. Vérification du fonctionnement de l'application

L'application peut être testée via une requête HTTP :

```
http://<NODE_IP>:<NODEPORT>/health
```

La réponse attendue est :

```
{"status": "OK"}
```

Cela confirme que l'application est correctement déployée et accessible.

11. Nettoyage des ressources

Après utilisation, il est recommandé de supprimer les ressources cloud afin d'éviter des coûts inutiles :

```
terraform destroy
```

Conclusion

Ce chapitre a présenté les différentes étapes permettant de reproduire l'infrastructure DevOps mise en place dans ce projet. La procédure détaillée permet de recréer l'environnement complet de manière structurée et reproductible. Elle illustre l'intérêt de l'approche Infrastructure as Code et de l'automatisation dans la gestion des systèmes modernes.

Chapitre 6 : Validation, résultats et analyse critique

Introduction

Ce chapitre a pour objectif d'évaluer la solution mise en place à travers les différents résultats obtenus et d'en proposer une analyse critique. Il permet de vérifier la cohérence globale de l'architecture DevOps réalisée, tout en identifiant les points forts et les limites du projet. Cette démarche est essentielle afin de situer le niveau de maturité de la solution et de proposer des axes d'amélioration pertinents.

1. Résultats obtenus

Les résultats observés montrent que le projet atteint un niveau de cohérence technique satisfaisant à travers l'ensemble de ses composantes.

Sur le plan applicatif, l'API développée avec Flask fonctionne correctement et expose les différents endpoints attendus. Les tests unitaires exécutés avec pytest confirment la validité du comportement de l'application, ce qui garantit une base logicielle fiable.

Du point de vue de l'infrastructure, les fichiers Terraform permettent de provisionner automatiquement un environnement AWS composé de deux instances EC2 jouant respectivement les rôles de nœud master et de nœud worker. Cette infrastructure constitue une base solide pour l'hébergement d'un cluster Kubernetes.

La configuration du cluster est assurée par Ansible, qui automatise l'installation de K3s et la jonction du nœud worker au cluster. Cette étape permet d'obtenir un cluster fonctionnel sans intervention manuelle, illustrant l'intérêt de l'automatisation dans un contexte DevOps.

En ce qui concerne le déploiement, les manifests Kubernetes définissent correctement les ressources nécessaires, notamment le Deployment et le Service. L'application peut ainsi être déployée et exposée via un service de type NodePort.

Enfin, l'enchaînement global des différentes étapes — tests, construction de l'image, provisionnement de l'infrastructure, configuration du cluster et déploiement — démontre que le projet couvre plusieurs maillons essentiels de la chaîne DevOps. L'ensemble ne se limite pas à l'utilisation isolée d'un outil, mais propose une intégration cohérente des différentes technologies.

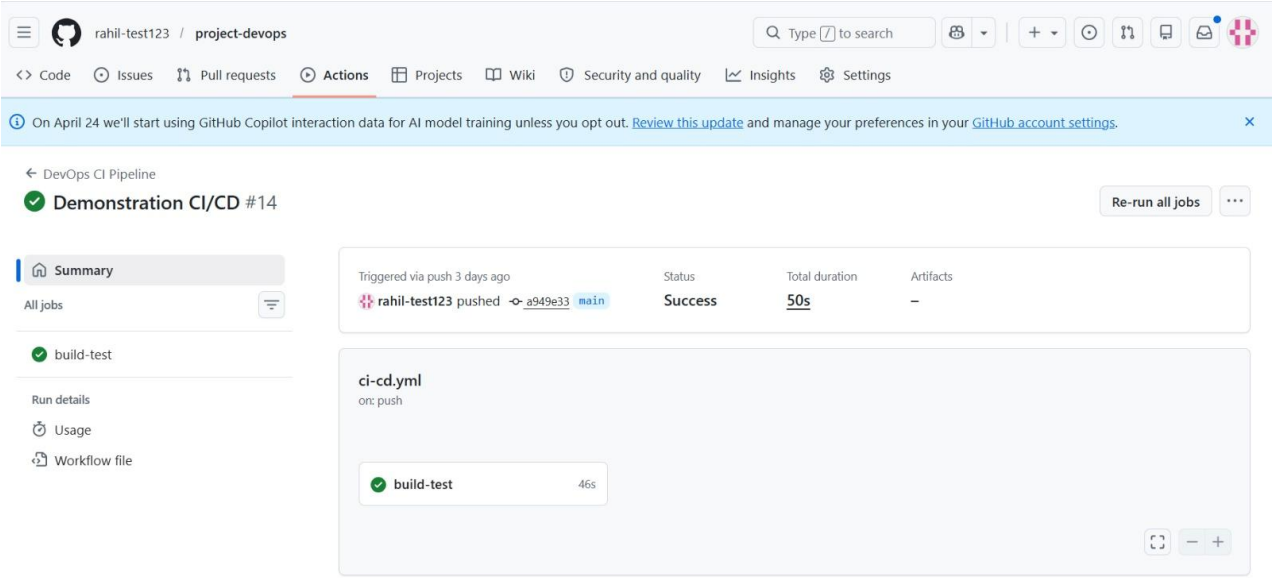

Figure 17-Exécution réussie du pipeline CI/CD sur GitHub Actions-

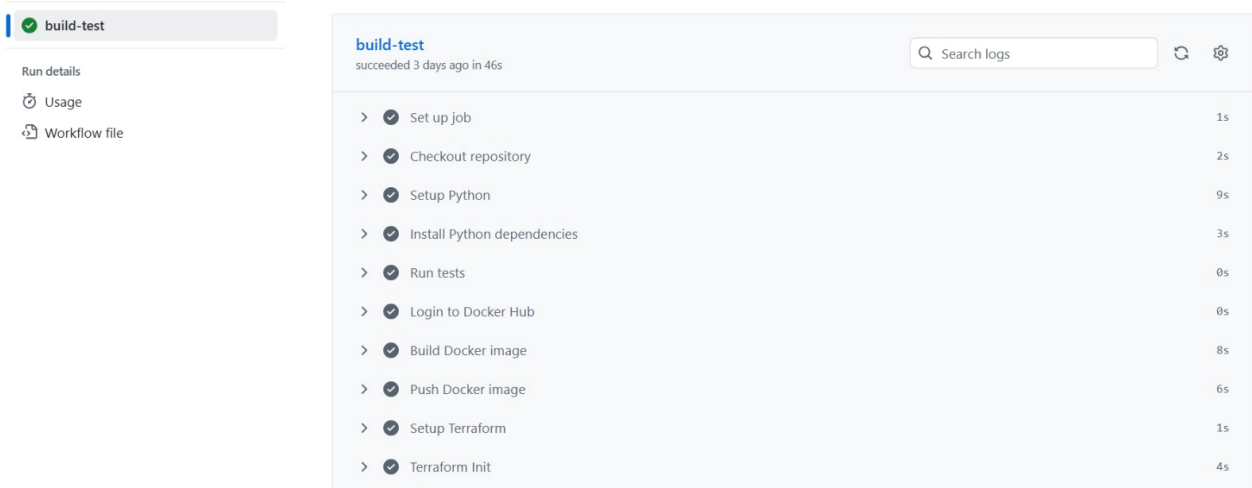

Figure 18-Détail des étapes du pipeline CI/CD

Figure 19-Détail des étapes du pipeline CI/CD

2. Forces du projet

Le projet présente plusieurs points forts qui témoignent d'une bonne compréhension des principes DevOps.

Tout d'abord, l'architecture mise en place est à la fois simple et complète. Elle permet de couvrir les principales étapes du cycle de vie d'une application, depuis son développement jusqu'à son déploiement sur une infrastructure cloud.

L'utilisation combinée de technologies telles que AWS, Terraform, Ansible, Docker, Kubernetes (K3s) et GitHub Actions constitue un choix pertinent. Ces outils sont largement utilisés dans l'industrie et leur intégration dans un même projet renforce la valeur technique de la solution.

La présence de tests unitaires intégrés dans le pipeline CI représente également un point positif important. Elle permet de garantir la qualité du code et d'automatiser la validation des fonctionnalités.

Le déploiement sur Kubernetes, basé sur un Deployment et un Service, montre une compréhension correcte des mécanismes d'orchestration. L'utilisation d'un service de type NodePort permet d'exposer l'application de manière simple et efficace.

Enfin, la documentation associée au projet, notamment les fichiers README et les étapes décrites dans le rapport, facilite la compréhension et la reproduction de la solution, ce qui constitue un élément essentiel dans un contexte professionnel.

3. Limites constatées

Le pipeline CI/CD mis en place permet d'automatiser les principales étapes du cycle de vie de l'application, notamment les tests, la construction de l'image Docker, sa publication sur Docker Hub ainsi que le déploiement sur Kubernetes.

Cette approche garantit une mise en production rapide et cohérente après chaque modification du code.

Néanmoins, certaines améliorations peuvent être envisagées afin d'optimiser davantage la solution, notamment en renforçant la gestion des versions des images Docker et en intégrant des outils complémentaires pour améliorer la gestion des déploiements.

4. Axes d'amélioration

Plusieurs améliorations peuvent être envisagées afin d'optimiser davantage la solution :

- Mettre en place un versionnement des images Docker
- Intégrer un environnement de staging
- Utiliser des outils comme Helm pour gérer les déploiements Kubernetes
- Ajouter une solution de monitoring (Prometheus, Grafana)

Conclusion

L'analyse des résultats montre que le projet répond globalement aux objectifs fixés et propose une implémentation cohérente d'une chaîne DevOps complète. Malgré certaines limitations, les fondations mises en place sont solides et démontrent une bonne maîtrise des outils et des concepts utilisés. Les axes d'amélioration identifiés ouvrent la voie à une évolution vers une solution plus automatisée, sécurisée et proche des standards professionnels.

Conclusion générale

À l'issue de ce projet, une chaîne DevOps fonctionnelle et cohérente a été mise en place, intégrant plusieurs outils et technologies complémentaires. L'ensemble des étapes, depuis le développement de l'application jusqu'à son déploiement sur Kubernetes, a été structuré de manière à illustrer concrètement les principes fondamentaux du DevOps.

La conteneurisation de l'application avec Docker a permis d'assurer la portabilité et la cohérence des environnements d'exécution. Le pipeline CI/CD, mis en œuvre via GitHub Actions, a permis d'automatiser les tests, la construction de l'image Docker, sa publication sur Docker Hub ainsi que le déploiement de l'application sur Kubernetes. Le recours à Terraform a facilité le provisionnement de l'infrastructure cloud de manière reproductible, tandis qu'Ansible a permis d'automatiser la configuration des machines et la création du cluster Kubernetes.

Le déploiement de l'application sur Kubernetes constitue une étape clé du projet, en assurant la gestion des conteneurs, leur scalabilité et leur accessibilité. L'ensemble de l'architecture démontre une bonne intégration des outils DevOps et une compréhension globale des différentes couches d'un système moderne.

Des améliorations peuvent toutefois être envisagées, notamment en optimisant la gestion des versions des images Docker et en intégrant des outils complémentaires pour la gestion avancée des déploiements et le monitoring de l'infrastructure.

En conclusion, ce projet constitue une base solide pour la mise en place d'environnements DevOps complets et met en évidence l'intérêt de l'automatisation dans la gestion du cycle de vie des applications. Il offre également des perspectives d'amélioration vers des architectures plus robustes, sécurisées et adaptées à des environnements de production.

Annexe A – Code source de l'application et Dockerfile

Cette annexe regroupe les fichiers applicatifs à joindre au rapport. Elle correspond exactement à l'exigence « Code source de l'application et Dockerfile ». Les contenus reproduits ci-dessous proviennent du projet remis.

A.1 – app.py

```
from flask import Flask, jsonify, render_template, request, send_from_directory

app = Flask(__name__, template_folder="templates", static_folder="static")

playlists = [
    {
        "id": 1,
        "title": "Chill Vibes",
        "cover": "image/chillvibes.jpg",
        "first_song": "Keep The Rain",
        "audio_url": "/static/audio/keep_the_rain.mp3"
    },
    {
        "id": 2,
        "title": "Focus Mode",
        "cover": "image/focusmode.jpg",
        "first_song": "On The Nature Of Daylight",
        "audio_url": "/static/audio/on_the_nature_of_daylight.mp3"
    },
    {
        "id": 3,
        "title": "Party Time",
        "cover": "image/partytime.jpg",
        "first_song": "Sugar On My Tongue",
        "audio_url": "/static/audio/sugar_on_my_tongue.mp3"
    },
    {
        "id": 4,
        "title": "Late Night",
        "cover": "image/latenight.jpg",
        "first_song": "Male Fantasy",
        "audio_url": "/static/audio/male_fantasy.mp3"
    }
]

albums = [
    {
        "id": 1,
        "title": "Eternal Sunshine",
        "artist": "Ariana Grande",
        "cover": "image/eternalsunshine.jpg",
        "first_song": "intro (end of the world)"
    },
    {
        "id": 2,
        "title": "Happier Than Ever",
```

```
        "artist": "Billie Eilish",
        "cover": "image/happierthanever.jpg",
        "first_song": "Getting Older"
    },
    {
        "id": 3,
        "title": "Amir",
        "artist": "Tamino",
        "cover": "image/amir.jpg",
        "first_song": "Habibi"
    },
    {
        "id": 4,
        "title": "Ultraviolence",
        "artist": "Lana Del Rey",
        "cover": "image/ultraviolence.jpg",
        "first_song": "Cruel World"
    }
]

songs = [
    {
        "id": 1,
        "title": "House Song",
        "artist": "Searows",
        "cover": "image/housesong.jpg"
    },
    {
        "id": 2,
        "title": "WILDFLOWER",
        "artist": "Billie Eilish",
        "cover": "image/hitmehard.png"
    },
    {
        "id": 3,
        "title": "i love you",
        "artist": "Billie Eilish",
        "cover": "image/iloveyou.jpg"
    },
    {
        "id": 4,
        "title": "Strangers",
        "artist": "Ethel Cain",
        "cover": "image/strangers.jpg"
    }
]

top_albums = [
    {
        "title": "HIT ME HARD AND SOFT",
        "artist": "Billie Eilish",
        "cover": "image/hitmehard.png"
    },
    {
        "title": "OK Computer",
        "artist": "Radiohead",
        "cover": "image/Okcomputer.png"
    },
```

```
    {
        "title": "Good Riddance",
        "artist": "Gracie Abrams",
        "cover": "image/Goodriddance.png"
    }
]

top_artists = [
    {
        "name": "Billie Eilish",
        "genre": "Alternative Pop",
        "cover": "image/billieeilish.png"
    },
    {
        "name": "Deftones",
        "genre": "Alternative Metal",
        "cover": "image/deftones1.jpg"
    },
    {
        "name": "The Beatles",
        "genre": "Rock / Pop Rock",
        "cover": "image/beatles.jpg"
    }
]




@app.route("/")
def home():
    return render_template("index.html")

@app.route("/browse")
def browse():
    return render_template("page1.html")

@app.route("/page1.html")
def browse_legacy():
    return render_template("page1.html")



@app.route("/image/<path:filename>")
def serve_image(filename):
    return send_from_directory("static/image", filename)



@app.route("/api/playlists", methods=["GET"])
def get_playlists():
    return jsonify(playlists)

@app.route("/api/albums", methods=["GET"])
def get_albums():
    return jsonify(albums)

@app.route("/api/songs", methods=["GET"])
def get_songs():
```

```
    return jsonify(songs)

@app.route("/api/home", methods=["GET"])
def get_home_data():
    return jsonify({
        "top_albums": top_albums,
        "top_artists": top_artists
    })

@app.route("/api/player/<category>/<int:item_id>", methods=["GET"])
def get_player_item(category, item_id):
    source_map = {
        "playlists": playlists,
        "albums": albums,
        "songs": songs
    }

    if category not in source_map:
        return jsonify({"error": "Invalid category"}), 400

    item = next((x for x in source_map[category] if x["id"] == item_id), None)

    if not item:
        return jsonify({"error": "Item not found"}), 404

    if category == "playlists":
        payload = {
            "title": item["title"],
            "cover": item["cover"],
            "subtitle": item["first_song"],
            "audio_url": item.get("audio_url")
        }
    elif category == "albums":
        payload = {
            "title": item["title"],
            "cover": item["cover"],
            "subtitle": item["first_song"],
            "audio_url": None
        }
    else:
        payload = {
            "title": item["title"],
            "cover": item["cover"],
            "subtitle": item["artist"],
            "audio_url": None
        }

    return jsonify(payload)

@app.route("/api/search", methods=["GET"])
def search():
    query = request.args.get("q", "").strip().lower()

    if not query:
        return jsonify({
            "query": query,
            "playlists": playlists,
            "albums": albums,
```

```
            "songs": songs
        })

    filtered_playlists = [
        p for p in playlists
        if query in p["title"].lower() or query in p["first_song"].lower()
    ]

    filtered_albums = [
        a for a in albums
        if query in a["title"].lower()
        or query in a["artist"].lower()
        or query in a["first_song"].lower()
    ]

    filtered_songs = [
        s for s in songs
        if query in s["title"].lower() or query in s["artist"].lower()
    ]

    return jsonify({
        "query": query,
        "playlists": filtered_playlists,
        "albums": filtered_albums,
        "songs": filtered_songs
    })


@app.route("/health")
def health():
    return jsonify({
        "status": "OK"
    })

@app.route("/info")
def info():
    return jsonify({
        "project": "SoundWave DevOps Cloud Project",
        "version": "2.0",
        "frontend": True,
        "backend": "Flask",
        "features": [
            "Landing page",
            "Browse page",
            "Music playlists",
            "Albums",
            "Songs",
            "Search API",
            "Player API",
            "Audio playback for playlists"
        ]
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

A.2 – test_app.py

```
from app import app

def test_home():
    client = app.test_client()
    response = client.get("/")
    assert response.status_code == 200

def test_browse():
    client = app.test_client()
    response = client.get("/browse")
    assert response.status_code == 200

def test_health():
    client = app.test_client()
    response = client.get("/health")
    assert response.status_code == 200
    data = response.get_json()
    assert data["status"] == "OK"

def test_info():
    client = app.test_client()
    response = client.get("/info")
    assert response.status_code == 200
    data = response.get_json()
    assert data["project"] == "SoundWave DevOps Cloud Project"

def test_api_playlists():
    client = app.test_client()
    response = client.get("/api/playlists")
    assert response.status_code == 200
    data = response.get_json()
    assert isinstance(data, list)
    assert len(data) > 0
    assert "title" in data[0]

def test_api_albums():
    client = app.test_client()
    response = client.get("/api/albums")
    assert response.status_code == 200
    data = response.get_json()
    assert isinstance(data, list)
    assert len(data) > 0
    assert "artist" in data[0]

def test_api_songs():
    client = app.test_client()
    response = client.get("/api/songs")
    assert response.status_code == 200
    data = response.get_json()
    assert isinstance(data, list)
    assert len(data) > 0
    assert "title" in data[0]

def test_api_player_playlist():
    client = app.test_client()
    response = client.get("/api/player/playlists/1")
```

```
    assert response.status_code == 200
    data = response.get_json()
    assert data["title"] == "Chill Vibes"
    assert "audio_url" in data

def test_api_search():
    client = app.test_client()
    response = client.get("/api/search?q=billie")
    assert response.status_code == 200
    data = response.get_json()
    assert "albums" in data
    assert "songs" in data
```

A.3 – requirements.txt

```
Flask

pytest
```

A.4 – Dockerfile

```
FROM python:3.10-slim


WORKDIR /app


COPY app/requirements.txt .


RUN pip install -r requirements.txt


COPY . .


EXPOSE 5000


CMD ["python", "app.py"]
```

Annexe B – Configuration du pipeline CI/CD

Cette annexe contient le workflow GitHub Actions utilisé pour l'intégration continue. Dans la version actuelle, il couvre l'installation des dépendances, les tests et la construction de l'image Docker.

B.1 – .github/workflows/ci-cd.yml

```
name: DevOps CI/CD Pipeline

on:
  push:
    branches: [ "main" ]

jobs:
  build-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.9"

      - name: Install Python dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r app/requirements.txt

      - name: Run tests
        run: |
          cd app
          pytest test_app.py

      - name: Login to Docker Hub
        run: echo "${{ secrets.DOCKER_PASSWORD }}" | docker login -u "${{
secrets.DOCKER_USERNAME }}" --password-stdin

      - name: Build Docker image
        run: docker build -t rahil0212/devops-api:latest .

      - name: Push Docker image
        run: docker push rahil0212/devops-api:latest

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3

      - name: Terraform Init
        run: |
          cd terraform
          terraform init

      - name: Terraform Validate
        run: |
          cd terraform
          terraform validate
```

```
      - name: Setup SSH
        run: |
          mkdir -p ~/.ssh
          echo "${{ secrets.SSH_PRIVATE_KEY }}" > ~/.ssh/id_rsa
          chmod 600 ~/.ssh/id_rsa

      - name: Copy Kubernetes manifests
        run: |
          scp -o StrictHostKeyChecking=no -i ~/.ssh/id_rsa -r k8s ubuntu@${{ secrets.MASTER_IP
}}:/home/ubuntu/

      - name: Deploy to Kubernetes
        run: |
          ssh -o StrictHostKeyChecking=no -i ~/.ssh/id_rsa ubuntu@${{ secrets.MASTER_IP }} <<
'EOF'
            export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
            sudo kubectl apply -f /home/ubuntu/k8s/
            sudo kubectl rollout restart deployment devops-api
          EOF
```

Annexe C – Scripts Terraform

Cette annexe regroupe les fichiers Terraform de provisionnement. Le contenu ci-dessous est reproduit tel qu'il apparaît dans le projet.

C.1 – terraform/main.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "eu-west-3"
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

resource "aws_security_group" "devops_sg" {
  name        = "devops-sg"
  description = "Security group for DevOps project"
```

```
ingress {
  description = "SSH"
  from port   = 22
  to_port     = 22
  protocol    = "tcp"
  cidr blocks = ["0.0.0.0/0"]
}

ingress {
  description = "Kubernetes API"
  from port   = 6443
  to port     = 6443
  protocol    = "tcp"
  cidr blocks = ["0.0.0.0/0"]
}

ingress {
  description = "Kubelet"
  from_port   = 10250
  to port     = 10250
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

ingress {
  description = "Flannel VXLAN"
  from_port   = 8472
  to port     = 8472
  protocol    = "udp"
  cidr_blocks = ["0.0.0.0/0"]
}

ingress {
  description = "NodePort range"
  from_port   = 30000
```

```
    to_port     = 32767
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "All traffic between nodes"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    self        = true
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "master" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = "t3.small"
  vpc_security_group_ids = [aws_security_group.devops_sg.id]
  key_name               = "devops-key"

  tags = {
    Name = "k8s-master"
  }
}

resource "aws_instance" "worker" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = "t3.small"
```

```
  vpc_security_group_ids = [aws_security_group.devops_sg.id]
  key_name               = "devops-key"

  tags = {
    Name = "k8s-worker"
  }
}
```

C.2 – terraform/variables.tf

```
variable "aws_region" {
  default = "eu-west-3"
}

variable "key_name" {
  default = "devops-key"
}

variable "master_instance_type" {
  default = "t3.small"
}

variable "worker_instance_type" {
  default = "t3.small"
}
```

C.3 – terraform/outputs.tf

```
output "master_public_ip" {
  value = aws_instance.master.public_ip
}

output "worker_public_ip" {
  value = aws_instance.worker.public_ip
}
```

```
output "master private ip" {
  value = aws_instance.master.private_ip
}

output "worker_private_ip" {
  value = aws_instance.worker.private_ip
}
```

Annexe D – Playbooks Ansible

Cette annexe présente l'inventaire et le playbook de configuration du cluster K3s. Les adresses IP ont été anonymisées dans le texte du rapport par des marqueurs génériques afin d'éviter l'exposition inutile d'informations d'infrastructure.

D.1 – ansible/inventory.ini (anonymisé)

```
[master]

master ansible host=<MASTER PUBLIC IP>


[worker]

worker ansible host=<WORKER PUBLIC IP>


[all:vars]

ansible user=ubuntu

ansible_ssh_private_key_file=../keys/devops-key.pem
```

D.2 – ansible/install-k3s.yml (anonymisé)

```
- name: Install K3s on master

  hosts: master

  become: yes

  tasks:

    - name: Install K3s server

      shell: curl -sfL https://get.k3s.io | sh -


- name: Get K3s token from master

  hosts: master

  become: yes

  tasks:

    - name: Read node token

      command: cat /var/lib/rancher/k3s/server/node-token

      register: token


    - name: Set token fact

      set_fact:

        k3s_token: "{{ token.stdout }}"
```

```
- name: Install K3s on worker
  hosts: worker
  become: yes
  tasks:
    - name: Install K3s agent
      shell: |
        curl -sfL https://get.k3s.io | \
        K3S_URL=https://<MASTER_PRIVATE_IP>:6443 \
        K3S_TOKEN={{ hostvars['master'].k3s_token }} sh -
```

Annexe E – Manifests Kubernetes

Cette annexe reprend les manifests Kubernetes utilisés pour le déploiement de l'application sur le cluster K3s.

E.1 – k8s/deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: devops-api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: devops-api
  template:
    metadata:
      labels:
        app: devops-api
    spec:
      containers:
        - name: devops-api
          image: rahil0212/devops-api:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 5000
```

E.2 – k8s/service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: devops-api-service
spec:
  selector:
    app: devops-api
  ports:
    - protocol: TCP
```

```
      port: 80

      targetPort: 5000

  type: NodePort
```

Annexe G – Documentation opératoire synthétique

G.1 – Procédure courte de reproduction

```
1) Provisionner les ressources AWS
   - cd terraform
   - terraform init
   - terraform plan
   - terraform apply

2) Relever les sorties Terraform
   - terraform output master_public_ip
   - terraform output worker_public_ip
   - terraform output master private ip

3) Mettre à jour l'inventaire et le playbook Ansible
   - renseigner <MASTER_PUBLIC_IP> et <WORKER_PUBLIC_IP> dans inventory.ini
   - renseigner <MASTER_PRIVATE_IP> dans install-k3s.yml si nécessaire

4) Vérifier l'accès puis installer K3s
   - ansible all -i inventory.ini -m ping
   - ansible-playbook -i inventory.ini install-k3s.yml

5) Vérifier le cluster
   - ssh -i ./keys/devops-key.pem ubuntu@<MASTER_PUBLIC_IP>
   - sudo k3s kubectl get nodes -o wide

6) Tester et conteneuriser l'application
   - cd app
   - pip install -r requirements.txt
   - pytest
   - docker build -t devops-api .
   - docker tag devops-api rahil0212/devops-api:latest
   - docker push rahil0212/devops-api:latest

7) Déployer sur Kubernetes
   - sudo k3s kubectl apply -f k8s/deployment.yaml
   - sudo k3s kubectl apply -f k8s/service.yaml
   - sudo k3s kubectl get pods
   - sudo k3s kubectl get svc

8) Nettoyer l'environnement
   - terraform destroy
```